

XiaoLee — Documentação Técnica para Investidores

XiaoLee — Documentação Técnica para Investidores

AI Ops Layer não-custodial para a economia de criadores sobre Stellar

Sumário

1. Executive Summary
 2. Produto e Estado Atual
 - 2.1 O que a XiaoLee entrega
 - 2.2 Diferenciais e impacto gerado
 - 2.3 Público-alvo
 - 2.4 Estado de produto (honesto)
 3. Prova Técnica Verificável
 - 3.1 Track Solana — contrato em devnet
 - 3.2 Track Stellar — fluxo x402 em testnet
 - 3.3 Auditoria de segurança interna (Sprint 2026-05)
 4. Arquitetura e Fluxos Críticos
 - 4.1 Arquitetura de alto nível
 - 4.2 Ciclo de vida de uma campanha
 - 4.3 Fluxo 1 — Autenticação SEP-10 (não-custodial)
 - 4.4 Fluxo 2 — Swap conversacional via Stellar DEX
 - 4.5 Fluxo 3 — Micropagamento de AI (x402)
 - 4.6 Fluxo 4 — Campanha: join / verify / claim (Stellar)
 - 4.7 Contrato Soroban xiao_lee_core (especificado, P0)
 5. Segurança e Modelo de Custódia
 6. Modelo de Negócio e GTM
 - 6.1 Modelo de receita
 - 6.2 Canais de aquisição
 - 6.3 Síntese de posicionamento
 7. Roadmap e Investimento Pleiteado
 - 7.1 Status por fase (37 Graus, Track A)
 - 7.2 Marcos críticos (gates)
 - 7.3 Investimento pleiteado
 8. Riscos e Mitigações
- Anexo A — Decisões de Produto (PDRs)
- Anexo B — Decisões de Arquitetura (ADRs)
- Anexo C — Referência de Endpoints e Configuração
- C.1 Endpoints (backend FastAPI)
 - C.2 Configuração (valores de teste)
 - C.3 Stack tecnológico

XiaoLee — Documentação Técnica para Investidores

AI Ops Layer não-custodial para a economia de criadores sobre Stellar

Versão 1.0 — 2026-05-30 · Programa 37 Graus (NearX/SDF), Track A · Stellar Testnet (MVP em construção, base técnica auditada) Builder: Gustavo Fontes (f0ntz) · fontzweb3@gmail.com · ParaDevs (Brazil) Repositório: github.com/ParaDevs-Brasil/XiaoLee

Sumário

- [1. Executive Summary](#)
 - [2. Produto e Estado Atual](#)
 - [3. Prova Técnica Verificável](#)
 - [4. Arquitetura e Fluxos Críticos](#)
 - [5. Segurança e Modelo de Custódia](#)
 - [6. Modelo de Negócio e GTM](#)
 - [7. Roadmap e Investimento Pleiteado](#)
 - [8. Riscos e Mitigações](#)
 - [Anexo A — Decisões de Produto \(PDRs\)](#)
 - [Anexo B — Decisões de Arquitetura \(ADRs\)](#)
 - [Anexo C — Referência de Endpoints e Configuração](#)
-

1. Executive Summary

A XiaoLee é uma **AI ops layer não-custodial** para a economia de criadores: uma camada de inteligência conversacional que senta entre o usuário e a infraestrutura Stellar e orquestra operações on-chain — swaps, campanhas sociais, distribuição de recompensas e pagamentos — em linguagem natural, via Twitter DM, Telegram ou web. O usuário nunca precisa entender carteiras, slippage, trustlines ou contratos: descreve a intenção, a XiaoLee prepara a transação e a assinatura permanece sempre na carteira do usuário.

Problema. A economia de criadores cresce sobre rails de pagamento que não a servem: cartão e plataformas centralizadas bloqueiam contas sem aviso, aplicam chargebacks que destroem margem e cobram 20–50% de comissão. No Brasil e na LATAM, o fã sem cartão internacional simplesmente não consegue pagar. As soluções on-chain endereçam parte disso, mas exigem que o usuário aprenda DeFi — uma barreira que o criador e o seu público não absorvem.

Solução. A XiaoLee transforma a interface do DeFi Stellar em uma conversa. Três pilares — engine de campanhas sociais, DeFi conversacional (swaps e pagamentos via Stellar DEX) e gateway de entrada/saída em Pix para a LATAM — operam sobre um princípio único: **wallet-first e non-custodial**. O backend orquestra, prepara e roteia; a chave privada nunca toca o servidor. A camada blockchain permanece transparente para o usuário, e o vínculo social-financeiro (Twitter ID como identidade) é o que diferencia o protocolo de uma carteira genérica.

Estado em 30/05/2026. O produto é um **MVP em construção com base técnica sólida e auditada internamente**, não um conceito. A camada Stellar off-chain — autenticação SEP-10, micropagamentos x402, cotação e montagem de swaps via Stellar DEX, âncora SEP-24 e integração Freightier no frontend — está implementada e passou por auditoria de segurança interna (23 findings corrigidos, §3.3). O contrato on-chain da track Solana já opera em devnet; o contrato Soroban da track Stellar está especificado e é o próximo marco (P0). A XiaoLee **ainda não opera em mainnet** e este documento não reporta tração de usuários — reporta o que é verificável hoje no código e on-chain em rede de teste (§3).

Categoria	AI ops layer não-custodial para economia de criadores (campanhas + DeFi conversacional + on/off ramp)
Blockchain	Stellar (testnet, direção estratégica do 37 Graus) + Soroban; track Solana/Anchor em devnet (legado multi-chain)
Estágio	MVP em construção · camada off-chain auditada · contrato Soroban a escrever · Track A 37 Graus
Prova verificável	Programa Solana em devnet (§3.1); fluxo x402 em Stellar testnet (§3.2); auditoria interna com 23 findings corrigidos e 61 testes passando (§3.3)
Diferencial	Interface em linguagem natural · non-custodial · Pix nativo via EtherFuse · micropagamentos agentic x402 · identidade social (Twitter ID)
Modelo de receita	Fee de swap (0,3%) + fee de campanha (0,5%) + x402 premium por query de AI (ativação em mainnet, §6.1)
Investimento pleiteado	Aplicação ao SCF Build Award (Stellar Community Fund); montante detalhado em documento SCF dedicado (§7.3)

2. Produto e Estado Atual

2.1 O que a XiaoLee entrega

Uma interface de AI conversacional para o DeFi Stellar, construída sobre três pilares. Cada pilar endereça uma fricção concreta que hoje limita a adoção de cripto por criadores e pelo seu público.

1. Engine de Campanhas sociais com recompensa on-chain. Criadores publicam campanhas com tarefas sociais (seguir, repostar, comentar) e definem recompensas em \$XLEE, XLM ou USDC. O usuário completa as tarefas, a XiaoLee verifica e a recompensa é distribuída diretamente na carteira Freightier — sem código e sem custódia.

Impacto: o criador monta um programa de engajamento pago em minutos, sem desenvolver contrato próprio, e constrói uma audiência on-chain que nenhuma plataforma centralizada pode remover ou bloquear.

2. DeFi conversacional non-custodial. Swaps, consultas de saldo e envio de pagamentos por mensagem. “Troca 50 USDC por XLM” → a XiaoLee consulta o melhor caminho no Stellar DEX, apresenta o quote, o usuário confirma e o Freightier assina. O backend monta a transação não assinada (XDR); a chave nunca sai da carteira.

Impacto: elimina a curva técnica do DeFi. O usuário opera a liquidez nativa da camada-1 da Stellar sem aprender interface de DEX, sem aprovar contrato de terceiro e sem expor a chave privada.

3. Gateway de pagamentos Pix/LATAM. On/off ramp via **EtherFuse** (parceiro oficial do programa 37 Graus): o usuário entra com Pix, opera no DeFi Stellar e sai quando quiser, sem comprar cripto em exchange. A EtherFuse abstrai SEP-24, KYC e a conversão fiat ↔ Stellar.

Impacto: desloca a barreira de entrada do DeFi do “tenha uma exchange e um cartão internacional” para “tenha um celular e um CPF”. É o caminho Pix → DeFi que hoje não existe de forma conversacional no Brasil.

2.2 Diferenciais e impacto gerado

Capacidade	Impacto gerado
Linguagem natural como única interface	Remove a curva técnica do DeFi: o criador e o fã operam por conversa, encurtando o onboarding a uma mensagem.
Non-custodial por design (chave nunca toca o backend)	Um eventual comprometimento do servidor não movimenta fundos — superfície de ataque substancialmente menor e <i>due diligence</i> mais simples.
Identidade social (Twitter ID como âncora)	A campanha recompensa “um criador com audiência verificada”, não “uma carteira” — a wallet pode ser trocada; a identidade social, não (PDR-001).
Pix nativo via EtherFuse	Único caminho conversacional Pix ↔ DeFi para a LATAM — abre o funil para o público sem cartão internacional.
Micropagamentos agentic via x402	A própria AI cobra por query premium em XLM (HTTP 402), criando um modelo de receita on-chain nativo para a economia de agentes.
Multi-chain isolado (Solana + Stellar)	A camada de orquestração e o DB são compartilhados (coluna chain); a integração Stellar não regrida os rails Solana já em devnet.
Custo de rede em frações de centavo (Stellar L1)	Viabiliza recompensa pequena e micropagamento de AI sem erosão de margem por gás.

2.3 Público-alvo

Segmento	Dor central	Solução da XiaoLee
Criadores LATAM e globais (Twitter/X, streamers, artistas)	Sem ferramenta para monetizar engajamento on-chain sem exigir que o fã configure carteira do zero	Engine de campanhas: tarefa social → recompensa na carteira, sem código e sem custódia
Fãs / usuários crypto-nativos (18–35)	Cada protocolo tem uma UX diferente; querem uma conversa só, em qualquer língua	Interface única em linguagem natural sobre o DeFi Stellar
Usuários não-crypto (LATAM, Pix)	Barreira de entrada ao DeFi via rail de pagamento local	Pix nativo via EtherFuse — entra com R\$ e opera no DeFi sem exchange
Projetos Web3 globais	Custo alto de desenvolvimento de campanhas com recompensa on-chain	Plataforma escalável de campanhas como serviço, independente de região

2.4 Estado de produto (honesto)

A XiaoLee é um MVP em construção: a base do protocolo (backend, campanhas, AI, observabilidade) está funcional e a camada Stellar off-chain foi auditada internamente. Este documento **não reporta métricas de tração de usuários** — o produto não está em mainnet e ainda não foi distribuído ao público final. O que é reportável hoje é a prova técnica verificável da §3.

Dimensão	Estado em 2026-05-30	Track
Camada Stellar off-chain (SEP-10, x402, swap DEX, SEP-24, Freighter)	Implementada e auditada	Stellar
Contrato on-chain	Em devnet (Anchor) / especificado e a escrever (Soroban)	Solana / Stellar
Backend API + DB multi-chain	Funcional; 61 testes passando; auditoria interna feita	Compartilhado
Canal conversacional	Telegram operacional; X/Twitter DM pendente de API paga	Compartilhado
Mainnet	Não — bloqueada por contrato Soroban + auditoria externa	Ambas

A XiaoLee não se apresenta como produto em escala: apresenta-se como uma camada de orquestração de AI com fundação técnica auditada, posicionada para destravar a track Stellar e ir a mainnet com segurança.

3. Prova Técnica Verificável

Em linha com o estágio real do produto, a prova abaixo é o que pode ser verificado hoje — em código, em rede de teste e na auditoria — e não envolve fundos reais em mainnet.

3.1 Track Solana — contrato em devnet

O programa on-chain da track Solana opera na devnet pública, demonstrando o padrão de registro on-chain que o contrato Soroban replicará na track Stellar.

Item	Valor (devnet)
Program ID	Fmmpn79Tij8fzYHg31ekZz4MmK9ArGzN59VogfcwhXiM
Código	solana-program/xiaolee_core/programs/xiaolee_core/src/lib.rs
IDL no frontend	frontend/src/idl/xiaolee_core.json
Instruções executadas	initialize_global, initialize_user, record_swap (serialização Borsh validada), pause_protocol
Pendência P0	record_swap com submissão real requer SOLANA_ADMIN_KEYPAIR_B58 em vault

3.2 Track Stellar — fluxo x402 em testnet

O fluxo de micropagamento agentic (x402) opera na **Stellar testnet**: a AI cobra 0,5 XLM por query premium, o cliente paga na testnet e o backend verifica a transação via Horizon antes de processar a resposta.

Item	Valor
Rede	STELLAR_NETWORK=testnet
Protocolo	x402 (HTTP 402 Payment Required) — POST /v1/ai/query
Preço por query	0,5 XLM (STELLAR_X402_PRICE_XLM, configurável)
Verificação	StellarAdapter.verify_payment(tx_hash, pay_to, min_amount) via Horizon
Anti-replay	tx_hash registrado após o uso — reuso rejeitado (correção SEC-001, §3.3)
Autenticação	SEP-10 challenge/response → JWT (/auth/stellar/challenge → /auth/stellar/token)
Swaps	Stellar DEX via path payments — quote em /stellar/swap/quote, XDR montado no backend

A integração StellarAdapter.record_reward ao contrato Soroban (distribuição de \$XLEE em campanha) é, hoje, off-chain/stub: torna-se on-chain quando o contrato da §4.6 for escrito e implantado (Gate 1B, §7).

3.3 Auditoria de segurança interna (Sprint 2026-05)

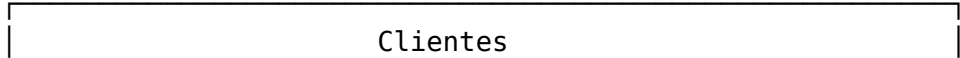
Auditoria de segurança interna conduzida sobre a branch develop, cobrindo backend (FastAPI), frontend (Next.js) e infraestrutura. **23 findings corrigidos**, com metodologia equivalente à de uma pré-auditoria de produção. Não substitui auditoria externa independente para mainnet com TVL real.

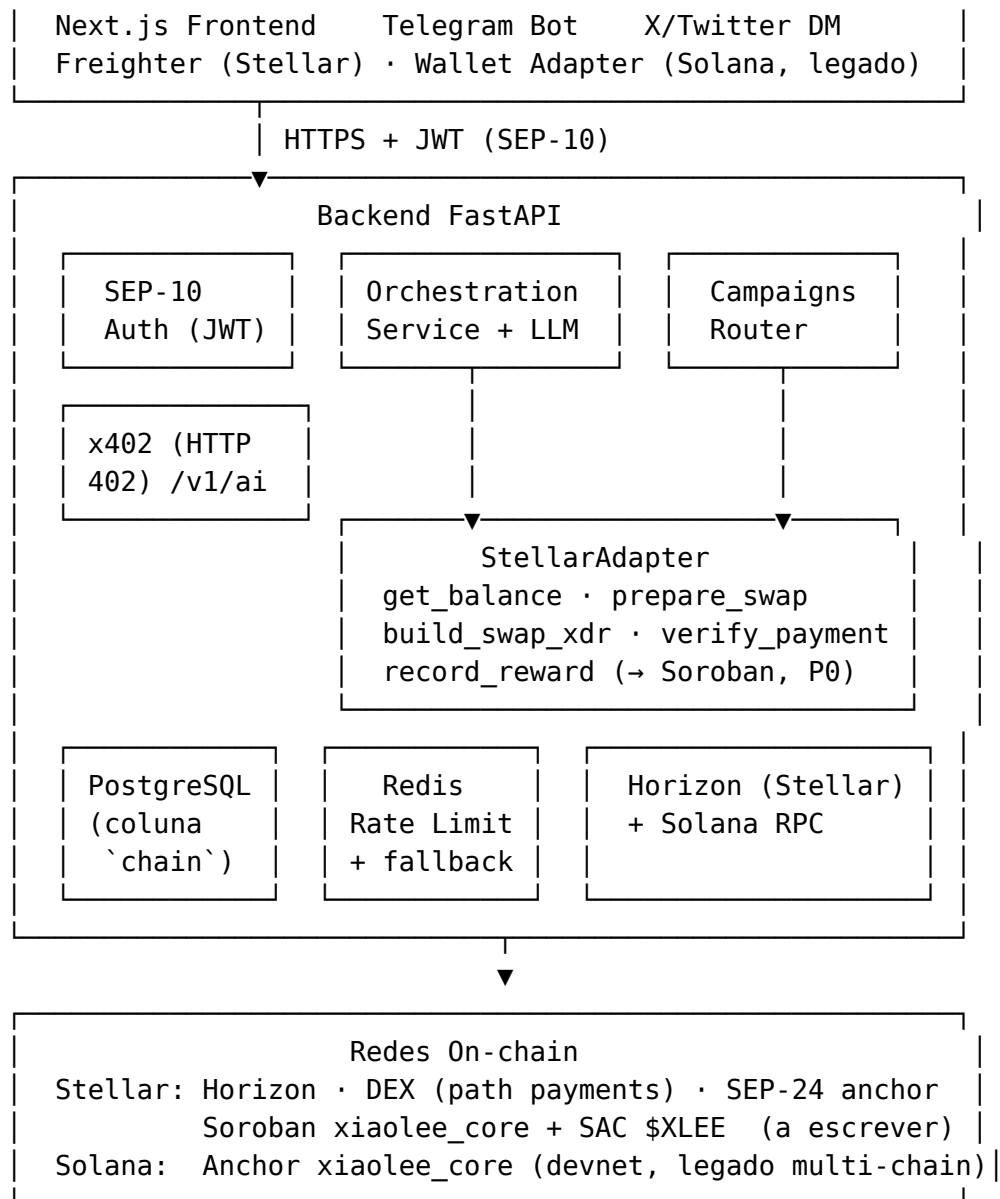
Severidade	Qtd	Status
Crítico	4	Corrigidos (SEC-001 replay x402, SEC-002 JWT_SECRET obrigatório, SEC-011, SEC-012)
Alto	9	Corrigidos (inclui SEP-10 SEC-014/015, Helius HMAC SEC-007/011)
Médio	7	Corrigidos
Baixo	3	Avaliados (1 corrigido, 1 false positive, 1 aceito por UX)

Frente	Cobertura
SAST	Semgrep 1.163.0 OSS — Python (security-audit, secrets, fastapi, OWASP Top 10, CWE Top 25, Trail of Bits), JS/TS (react, nextjs), Docker
Cripto / timing	Análise (metodologia Trail of Bits) em encryption_service.py, stellar_auth_routes.py, helius_routes.py, x402_routes.py
Supply chain	11 dependências flagradas (5 de alto risco); dependências mortas removidas; Dependabot configurado
Fuzzing	Hypothesis property-based — 15 testes, ~2.500 exemplos; encontrou e corrigiu 2 bugs reais
Carga	load_tests/locustfile.py — 5 classes, incluindo XiaoLeeStellarAuth (stress SEP-10) e replay/SQLi
Suíte	61 testes passando, 6 skipped

4. Arquitetura e Fluxos Críticos

4.1 Arquitetura de alto nível





4.2 Ciclo de vida de uma campanha

CREATED → ENROLLED → TASKS_VERIFIED → { REWARDED | EXPIRED }

O participante entra (enrolled), completa as tarefas sociais, a XiaoLee verifica (tasks_verified) e, na claim, a recompensa é distribuída e registrada (rewarded). Idempotência garantida por constraint única (uq_participant_campaign_user) — uma segunda inscrição retorna 409 Conflict.

4.3 Fluxo 1 — Autenticação SEP-10 (não-custodial)

1. Frontend: GET /auth/stellar/challenge?account=G...ABC
2. Backend: gera challenge XDR (sequence=0, manage_data op, nonce, TTL 5min)
3. Frontend: Freightier.sign(XDR) -> signed_xdr
4. Frontend: POST /auth/stellar/token { account, transaction: signed_xdr }
5. Backend: valida a assinatura com stellar-sdk e extrai a public key

6. Backend: emite JWT { sub: twitter_id, stellar_wallet: G...ABC, chain: "stellar" }
7. Frontend: armazena o JWT – todas as requests subsequentes o incluem

O backend mantém um keypair dedicado exclusivamente ao challenge SEP-10, nunca usado para fundos. A chave privada do usuário nunca toca o servidor (ADR-002).

4.4 Fluxo 2 — Swap conversacional via Stellar DEX

Usuário: "troca 50 USDC por XLM"

1. OrchestrationService: intent = stellar_swap
2. StellarAdapter.prepare_swap(USDC, XLM, 50)
 - GET /paths/strict-send (Horizon) -> melhor caminho + destination_amount
3. XiaoLee: "Você receberá ~142.3 XLM. Confirma?"
4. Usuário confirma
5. Backend: build_swap_xdr -> XDR unsigned (pathPaymentStrictSend)
6. Freightler: o usuário assina
7. Frontend submete via Horizon -> Backend persiste swap_record (chain="stellar")

A liquidez é a nativa da camada-1 da Stellar — sem contrato de DEX de terceiro e sem aprovação de token (ADR-003).

4.5 Fluxo 3 — Micropagamento de AI (x402)

1. Cliente: POST /v1/ai/query (sem pagamento)
2. Backend: HTTP 402 Payment Required + payment-info { asset: XLM, amount: 0.5, pay_to }
3. Cliente: paga 0.5 XLM na Stellar testnet (Freightler) -> tx_hash
4. Cliente: POST /v1/ai/query com header X-Payment { tx_hash, network: testnet }
5. Backend: verify_payment(tx_hash, pay_to, 0.5) via Horizon
 - rejeita se o hash já foi usado (anti-replay, SEC-001)
6. Backend: processa a query e retorna a resposta da AI

É o caso de uso *agentic*: a própria AI cobra por uso premium, em XLM, de forma nativa ao protocolo HTTP — modelo de receita on-chain sem intermediário (PDR-007).

4.6 Fluxo 4 — Campanha: join / verify / claim (Stellar)

```
POST /campaigns/join { campaign_id, stellar_wallet, chain: "stellar" }  
  -> participant enrolled (409 se já inscrito)  
Usuário completa as tarefas sociais (follow, RT, comment)  
POST /campaigns/verify { campaign_id, twitter_id, proof }  
  -> Backend verifica via API Twitter/X -> tasks_verified  
POST /campaigns/claim { campaign_id, stellar_wallet }  
  -> Backend valida o JWT (SEP-10)  
  -> StellarAdapter.record_reward(twitter_id, wallet, amount)  
      [hoje: registro off-chain | a escrever: invoca record_reward]
```

no Soroban + SAC \$XLEE]

-> persiste o receipt com tx_hash -> usuário recebe \$XLEE no Freighter

A etapa record_reward on-chain depende do contrato Soroban da §7 (Gate 1B). Até lá, a recompensa é registrada off-chain de forma auditável no banco.

4.7 Contrato Soroban xiaolee_core (especificado, P0)

Instrução	Descrição
initialize(admin, xlee_sac)	Inicializa o GlobalConfig (instance storage); require_auth
initialize_user(twitter_id)	Registra o usuário (persistent storage + bump de TTL)
record_reward(admin, twitter_id, amount)	Distribui \$XLEE via SAC; require_auth + checked_add (anti-overflow)
pause_protocol / unpause_protocol	Emergency pause
transfer_admin(admin, new_admin)	Migração de autoridade

Storage instance para config global e persistent por usuário; auth por Address::require_auth(); token via SAC (token::Client). Eventos reward_recorded, protocol_paused, admin_transferred. **Status: especificado no RT, código a escrever — é o marco que destrava a track Stellar.**

5. Segurança e Modelo de Custódia

A segurança é o núcleo do produto, não uma camada acessória. O princípio é abstrair a complexidade cripto do usuário sem nunca assumir o controle dos seus fundos. A tabela associa cada mecanismo ao risco concreto que ele elimina; o status reflete o estado real verificado no código.

Mecanismo	Risco eliminado	Status
Non-custodial — XDR montado no backend, assinado no Freighter	Um vazamento do servidor não move fundos: a chave do usuário nunca está sob a plataforma (ADR-002)	Implementado
Autenticação SEP-10 (challenge/response → JWT)	Sessão sem senha e sem custódia de chave; padrão Stellar compatível com âncoras	Implementado e auditado
Anti-replay no x402 (tx_hash registrado após uso)	Reuso de um pagamento para obter respostas de AT de graça (SEC-001)	Corrigido
JWT_SECRET obrigatório no startup (raise se ausente)	Segredo de assinatura fraco/default em produção (SEC-002)	Corrigido

Mecanismo	Risco eliminado	Status
Webhook Helius com HMAC SHA-256 validado	Injeção de eventos on-chain forjados (SEC-007/011)	Corrigido
Rate limiting por usuário (Redis + fallback in-memory)	Abuso e exaustão de recursos (ex.: flood de challenge SEP-10)	Implementado
Idempotência (constraint única → 409)	Inscrição/recompensa duplicada em campanha	Implementado
Verificação on-chain antes de creditar (Horizon)	Crédito baseado em estado local divergente do ledger	Implementado (x402/swap)
CORS restrito por env (sem wildcard)	Origem maliciosa consumindo a API autenticada	Implementado
Emergency pause no contrato	Continuidade de operação durante incidente on-chain	No Anchor (devnet); no Soroban via spec

Hardening pré-mainnet (BLOQUEADORES): auditoria externa independente dos contratos (Anchor + Soroban) e do backend; secrets em vault (keypairs, DB, Redis, JWT); HTTPS + HSTS; migração do admin EOA para multisig nativo (Squads na Solana / multisig Stellar); bug bounty em testnet pública. Nenhum deploy em mainnet com fundos reais ocorre antes desses gates (ver §3 do MAINNET_READINESS.md).

6. Modelo de Negócio e GTM

6.1 Modelo de receita

Fonte	Mecanismo	Ativação
Fee de swap	0,3% sobre o volume de swaps executados via XiaoLee	Mainnet (Fase 3)
Fee de campanha	0,5% sobre o volume de recompensas distribuídas	Mainnet (Fase 3)
x402 premium	Micropagamento em XLM por query de AI avançada	Testnet hoje (§3.2); mainnet em seguida

Diferentemente de fluxos B2B humano-para-humano, a XiaoLee abraça o x402 como receita nativa: o caso de uso *agentic* (a AI cobra por uso premium) é compatível com o produto e já está implementado em testnet — um vetor de monetização on-chain que não depende de intermediário de pagamento.

6.2 Canais de aquisição

Canal	Tática	Meta
Twitter/X (dogfooding)	Campanha self-referencial na própria plataforma: “complete tarefas → ganhe \$XLEE”	Primeiros usuários e prova de produto
Telegram	Bot XiaoLee com onboarding Stellar na comunidade (canal já operacional)	Base inicial conversacional
Criadores BR	Parceria com 3–5 influenciadores crypto BR para campanhas pagas	Liquidez de oferta nas campanhas
Comunidade Stellar BR	Presença em Telegram/Discord Stellar BR	Awareness
Build in public	Thread semanal com progresso e métricas reais	Credibilidade
Rede 37 Graus	Mentoria SDF e introduções no Rio	Pipeline e parceria EtherFuse

6.3 Síntese de posicionamento

Configure uma campanha em poucos minutos: seus fãs completam tarefas sociais e recebem a recompensa direto na carteira, sem código e sem custódia. Swaps e pagamentos por conversa, com a chave sempre no usuário. E, para a LATAM, o fã entra via Pix — sem exchange e sem cartão internacional. **A XiaoLee é a interface de AI para o DeFi Stellar, com Pix nativo.**

7. Roadmap e Investimento Pleiteado

7.1 Status por fase (37 Graus, Track A)

Programa NearX/SDF: 04/05–11/06/2026 · 5 Sprints · Residência no Rio 08–11/06.

Fase	Período	Status	Entregáveis-chave
1 — Infra Stellar + primeiros usuários	04–11/05	Em curso	StellarAdapter, SEP-10, Freighter no frontend, migração multi-chain (DB)
2 — Pagamentos: Pix, stablecoins, x402	11–18/05	Em curso	x402 em testnet (feito); EtherFuse on/off ramp (a

Fase	Período	Status	Entregáveis-chave
3 — DeFi e protocolo	18–25/05	Pendente	integrar); swap DEX Contrato Soroban + SAC \$XLEE; record_reward on-chain; campanhas verificáveis
4 — GTM e narrativa de investidor	25/05–01/06	Em curso	Deck, one-pager, este documento, vídeo demo
5 — Pitch Rio	08–11/06	Planejado	Apresentação presencial + demo ao vivo

7.2 Marcos críticos (gates)

Gate	Descrição	Status
1B — Contrato Soroban	Escrever xiaolee_core (auth, overflow check, eventos) + testes + deploy testnet	P0 — destrava a track Stellar
Asset \$XLEE (Stellar)	Emitir o asset + stellar contract asset deploy (SAC) em testnet	Pendente
Integração on-chain	Ligar StellarAdapter.record_reward ao contrato real (hoje off-chain)	Pendente
Auditoria externa	2 auditorias independentes por track + backend	BLOQUEADOR de mainnet
Secrets / vault + HTTPS/HSTS + multisig	Hardening de produção	BLOQUEADOR de mainnet

7.3 Investimento pleiteado

Aplicação ao **SCF Build Award** (Stellar Community Fund) para escrever e auditar o contrato Soroban, integrar o EtherFuse (Pix) e levar a track Stellar a mainnet com segurança. Montante e uso de recursos detalhados em documento SCF dedicado; submissão pendente.

Uso de recursos previsto:

- **Engenharia:** contrato Soroban xiaolee_core + SAC \$XLEE, integração on-chain do record_reward, integração EtherFuse.
- **Segurança:** auditoria externa por track + backend; migração de admin para multisig; secrets em vault.
- **Infraestrutura:** Horizon/Soroban RPC dedicados; provisionamento de PostgreSQL gerenciado e Redis em produção.

- **Comercial:** ativação dos primeiros criadores BR e do canal Telegram/X.

8. Riscos e Mitigações

Risco	Probabilidade	Impacto	Mitigação
Contrato Soroban não escrito a tempo (Gate 1B)	Média	Alto	Spec completa no RT (§4.7); contrato Anchor em devnet serve de referência de padrão; é o P0 priorizado
Ausência de auditoria externa para mainnet	Alta	Crítico	Mainnet bloqueada por gate; auditoria interna (23 findings) como pré-auditoria; candidatos mapeados (OtterSec, Veridise, Trail of Bits)
Dependência da API EtherFuse (Pix)	Média	Médio	Parceria oficial do 37 Graus; swap DEX nativo funciona sem o on/off ramp; integração isolada no StellarAdapter
X/Twitter DM exige API paga	Alta	Baixo	Telegram cobre 100% do fluxo conversacional; X é incremental, não bloqueia o hackathon
Complexidade do híbrido multi-chain (Solana + Stellar)	Média	Médio	Camada de orquestração e DB compartilhados (coluna chain); tracks isoladas, sem regressão mútua
Secrets/keypairs fora de vault	Alta (hoje)	Crítico (em mainnet)	Gate explícito antes de mainnet; em dev opera com chaves de teste e dry_run
Trustline \$XLEE como atrito de UX	Média	Médio	XiaoLee detecta a ausência e guia o usuário; o Freighter assina o trustline

Risco	Probabilidade	Impacto	Mitigação
			com um clique (ADR-005)

Anexo A — Decisões de Produto (PDRs)

PDR-001 — Twitter ID como identidade primária. A wallet Freighter é vinculada ao Twitter ID, não o contrário. *Justificativa:* o vínculo social-financeiro é o diferencial; uma campanha recompensa um criador com audiência verificada, não uma carteira. A wallet pode ser trocada; a identidade social, não.

PDR-002 — Freighter como wallet principal. Mantida pela SDF, com SDK TypeScript oficial e suporte nativo a Soroban. *Futuro:* xBull e LOBSTR via abstração WalletAdapter, sem quebrar a interface.

PDR-003 — On/off ramp via EtherFuse (Sprint 2). A EtherFuse (parceiro oficial do 37 Graus) abstrai SEP-24, KYC e a conversão fiat ↔ Stellar. *Justificativa:* reduz o escopo de implementação e entrega o diferenciador Pix sem integrar âncora diretamente.

PDR-004 — \$XLEE como Stellar Asset (SAC), não token Soroban puro. Liquidez nativa no Stellar DEX imediata; o SAC provê a interface para o contrato distribuir \$XLEE. Complexidade extra (vesting/locks) via wrapper Soroban quando necessário.

PDR-005 — SEP-10 para autenticação não-custodial. Padrão Stellar de challenge/response → JWT. *Justificativa:* compatibilidade com âncoras e integradores; suportado nativamente pelo Freighter.

PDR-006 — Soroban para o contrato XiaoLee Core. O registro on-chain de recompensa é diferencial de produto (auditabilidade sem confiar no backend). Rust/WASM com `require_auth()` e integração via SAC.

PDR-007 — x402 para micropagamentos de AI. A AI cobra em XLM por query premium (HTTP 402). *Justificativa:* modelo de receita on-chain nativo para a economia agentic — entregável explícito do 37 Graus, já em testnet.

PDR-008 — Track A no 37 Graus. A base do protocolo está funcional e auditada; a integração Stellar é expansão, não rebuild. Classificação coerente com o estágio real e competitiva nos critérios de escala.

Anexo B — Decisões de Arquitetura (ADRs)

ADR-001 — StellarAdapter como camada de abstração de protocolo. Encapsula Horizon, Soroban e EtherFuse; o OrchestrationService delega conforme a intenção detectada. *Rejeitado:* lógica Stellar dispersa nos controllers (acoplamento, sem teste isolado).

ADR-002 — XDR gerado no backend, assinado no frontend (non-custodial). O backend monta o XDR unsigned; o Freighter assina; o backend submete. As chaves do usuário nunca tocam o servidor.

ADR-003 — Stellar DEX (path payments) para swaps.
pathPaymentStrictSend/Receive via Horizon path finding. *Vantagem:* liquidez nativa da L1, sem contrato de terceiro nem aprovação de token.

ADR-004 — Soroban xiaolee_core com storage persistent e auth por Address.
instance para config global, persistent por usuário (TTL bumped pelo backend); Address::require_auth() sem workarounds; token via SAC.

ADR-005 — \$XLEE como Stellar Asset + SAC. Asset emitido + stellar contract asset deploy; o contrato usa o SAC para transferir. UX de trustline guiada pelo chat (assinatura em um clique no Freightier).

ADR-006 — PostgreSQL com coluna chain para eventos multi-origem. chain VARCHAR(16) em campaign_participants/swap_records e stellar_wallet em user_profiles; twitter_id permanece FK primária. Migração 20260515_stellar_columns.py.

ADR-007 — Horizon event_stream (SSE) para indexação on-chain. Task asyncio escuta transações via cursor now; eventos do contrato são processados e persistidos. *Rejeitado:* polling por cron (latência e custo).

Anexo C — Referência de Endpoints e Configuração

C.1 Endpoints (backend FastAPI)

Endpoint	Método	Responsabilidade
/auth/stellar/challenge	GET	Gera o challenge XDR (SEP-10)
/auth/stellar/token	POST	Valida o XDR assinado e emite o JWT
/stellar/swap/quote	GET	Quote de swap via Stellar DEX (path payment)
/stellar/anchor/info	GET	Info da âncora SEP-24
/stellar/anchor/challenge	GET	Challenge da âncora SEP-24
/stellar/anchor/deposit	POST	Inicia o depósito via âncora SEP-24
/v1/ai/query	POST	Query de AI premium com micropagamento x402
/v1/ai/query/payment-info	GET	Instruções de pagamento (asset, amount, pay_to)
/v1/ai/query/verify-tx	GET	Verifica o tx_hash do pagamento
/campaigns/join · /verify · /claim	POST	Ciclo de vida da campanha (idempotente, 409 em duplicidade)

Endpoint	Método	Responsabilidade
/metrics	GET	Prometheus
/health · /health/detailed	GET	Liveness e dependências (DB, Solana, Gemini)

C.2 Configuração (valores de teste)

```

STELLAR_NETWORK           = testnet
STELLAR_HORIZON_URL       = (configurável por env)
STELLAR_X402_PRICE_XLM   = 0.5
STELLAR_X402_ENABLED      = true
SOLANA_CLUSTER            = devnet
SOLANA_RPC_URL            = https://api.devnet.solana.com
SOLANA_PROGRAM_ID        =
Fmmpn79Tij8fzYHg3lekZz4MmK9ArGzN59VogfcwhXiM

```

C.3 Stack tecnológico

Camada	Tecnologia
Smart contract (Stellar)	Soroban SDK (Rust/WASM) — a escrever
Smart contract (Solana, legado)	Anchor (Rust) — devnet
Wallet	Freighter + @stellar/freighter-api
Swaps	Stellar DEX via Horizon path payments
Token	Stellar Asset XLEE + SAC (SEP-41)
Autenticação	SEP-10 challenge/response → JWT
Micropagamento de AI	x402 (HTTP 402, XLM)
On/off ramp BR/LATAM	EtherFuse API (parceiro 37 Graus) — a integrar
Backend	FastAPI (Python 3.12) + asyncio
Frontend	Next.js 15 (TypeScript)
Banco de dados	PostgreSQL 16 + Alembic async (coluna chain)
Cache / rate limit	Redis sliding window + fallback in-memory
Observabilidade	Prometheus /metrics + Grafana
Testes	61 passando · auditoria interna (23 findings corrigidos)

*XiaoLee — AI ops layer não-custodial para a economia de criadores sobre Stellar.
Programa 37 Graus — NearX/SDF · Track A · 2026 · MVP em construção, base técnica
auditada.*